41.2 TUI 交互机制

> **生成模型**：gpt-5.4（openai/gpt-5.4）
> **Token 消耗**：输入 ~22k tokens，输出 ~4k tokens（估算，本节）

如果说 33.1 讲的是 TUI 的骨架，那这一节要看的就是它怎么“动”起来。OpenClaw 的 TUI 不是把 WebSocket（WebSocket）事件原样吐到终端，而是先做一轮筛选、组装、去重，再把结果投喂给聊天区、状态栏和覆盖层。这样做有点像图形界面里的事件循环：外部世界不断抛来消息，前端再决定哪些该显示、哪些该丢掉、哪些该合并成一次稳定更新。

41.2.1 事件处理管线

TUI 侧最核心的两类事件是聊天事件（Chat Event）和代理事件（Agent Event）：

```
type ChatEvent = {
  runId: string;
  sessionKey: string;
  state: "delta" | "final" | "aborted" | "error";
  message?: unknown;
  errorMessage?: string;
};

type AgentEvent = {
  runId: string;
  stream: string;
  data?: Record<string, unknown>;
};
```

这两个类型都很短，但信息量不小。 `ChatEvent` 负责“这次对话跑到哪一步了”， `AgentEvent` 负责“这次运行内部又发生了什么”。前者偏用户可见，后者偏运行细节，尤其是工具调用。

整个处理管线大致是四步：先做会话同步检查（session sync check），再做去重（deduplication），然后做流组装（stream assembly），最后才是界面更新（UI update）。对应到 `src/tui/tui-event-handlers.ts` ，顺序非常明确：先 `syncSessionKey()` ，确认当前 TUI 还盯着同一个 `sessionKey` ；再检查 `finalizedRuns` ，避免已经结束的 `runId` 被重复刷屏；接着把 `delta` 片

段交给 `TuiStreamAssembler` 拼起来；拼出可显示文本后，才调用 `chatLog.updateAssistant()` 或 `chatLog.finalizeAssistant()` 。

```
if (!isSameSessionKey(evt.sessionKey, state.currentSessionKey)) {
  return;
}
if (finalizedRuns.has(evt.runId) && evt.state === "delta") {
  return;
}
const displayText = streamAssembler.ingestDelta(evt.runId, evt.message, state.showThink
if (displayText) {
  chatLog.updateAssistant(displayText, evt.runId);
}
```

这里还有一个很实用的运行生命周期（run lifecycle）管理。代码里同时维护 `sessionRuns` 和 `finalizedRuns` 两个 `Map` 。前者记录当前会话里还“活着”的运行，后者记录刚结束的运行，用来挡住网络抖动或延迟重放造成的重复事件。两个表都不是无限增长的：当条目超过 200 个时，系统会先尝试删除 10 分钟以前的旧记录；如果删完还超，就继续裁到 150 个左右。这个 150-200 的阈值，本质上是在内存占用和去重可靠性之间找平衡。

41.2.2 Gateway 事件处理

从 Gateway（网关）过来的聊天事件，分支处理很直接，但细节不少。

先看 `delta` 。它不代表一句完整回复，只代表“又多了一截”。这时 TUI 不会立刻落盘，也不会把消息视为完成，而是调用 `updateAssistant()` 做增量刷新，让终端里的回答像打字一样往外长。

`final` 才是真正的收口点。这里一般会先走 `streamAssembler.finalize()` ，得到最终可显示文本，再调用 `finalizeAssistant()` 固化到聊天区；如果这次运行属于本地发起，还会顺手触发 `loadHistory()` ，把会话历史重新拉一遍，确保终端里看到的内容和 Gateway 存储的转录（transcript）一致。

`aborted` 和 `error` 则是两个失败出口。前者通常显示取消信息，比如 `run aborted` ；后者显示 `run error: ...` 。它们都会清理运行状态，把 `activeChatRunId` 置空，并把活动状态改成 `aborted` 或 `error` 。也就是说，TUI 不会因为一次失败把整条界面链路卡死。

代理事件的重点在工具流（tool stream）。 `evt.stream === "tool"` 时，代码会按 `phase` 分三类： `start` 、 `update` 、 `result` 。这三类事件分别落到聊天日志的 `toolStart` 、 `toolUpdate` 、 `toolResult` 语义上。OpenClaw 在实现里用的是

`chatLog.startTool()` 和 `chatLog.updateToolResult()`，但从展示效果理解，把它看成“开始一条工具卡片、持续补内容、最后落最终结果”更容易。

```
if (phase === "start") {
  chatLog.startTool(toolCallId, toolName, data.args);
} else if (phase === "update") {
  chatLog.updateToolResult(toolCallId, data.partialResult, { partial: true });
} else if (phase === "result") {
  chatLog.updateToolResult(toolCallId, data.result, {
    isError: Boolean(data.isError),
  });
}
```

不过它还有一层“阀门”：只有 `verbose` 不是 `off` 时才显示工具事件；只有 `verbose === "full"` 时才显示增量工具输出。这个设计很像日志等级（log level）。普通用户看结果，高级用户看过程，两边都照顾到了。

41.2.3 命令系统详解

TUI 的斜杠命令（slash command）不是拿字符串硬比对，而是先经过 `parseCommand()` 统一解析。这个函数先去掉前导 `/`，再把第一段视为命令名，其余部分拼回参数字符串。比如 `/model openai/gpt-5`，最后会变成 `{ name: "model", args: "openai/gpt-5" }`。

```
export function parseCommand(input: string): ParsedCommand {
  const trimmed = input.replace(/^\//, "").trim();
  if (!trimmed) return { name: "", args: "" };
  const [name, ...rest] = trimmed.split(/\s+/);
  const normalized = name.toLowerCase();
  return {
    name: COMMAND_ALIASES[normalized] ?? normalized,
    args: rest.join(" ").trim(),
  };
}
```

`COMMAND_ALIASES` 目前至少定义了 `/elev -> /elevated`。这看着像个小优化，实际很重要：命令行交互里，缩短击键路径能明显降低摩擦。再往上看 `getSlashCommands()`，OpenClaw 内置的命令已经超过 25 个，除了题目里列出的 `/help`、`/status`、`/agent`、`/session`、`/model`、`/think`、`/verbose`、`/reasoning`、`/usage`、`/elevated`、`/activation`、`/abort`、`/new`、`/reset`、`/settings`、`/exit`、`/quit`

，还包括 `/agents` 、 `/sessions` 、 `/models` ，以及从 Gateway 命令注册表动态并入的一批聊天命令。

命令系统真正顺手的地方在参数自动补全（argument auto-completion）。例如 `/think` 会根据当前模型能力给出 `off` 、 `minimal` 、 `low` 、 `medium` 、 `high` 、 `xhigh` 等级。用户不用死记，按 Tab 就能试探系统支持什么。这个思路和 IDE 里的智能提示很像：不是让用户记协议，而是让协议自己暴露可选项。

41.2.4 会话操作

会话切换看起来只是换一个键，其实背后掺杂了作用域（scope）和 Agent 归属两个维度。 `resolveTuiSessionKey()` 就是这里的入口。它接收用户原始输入，如果用户没填内容，就根据当前模式返回默认键：全局模式（global）直接给 `global` ，按发送者分会话（per-sender）则生成 `agent:<id>:<mainKey>` 这样的键。

```
export function resolveTuiSessionKey(params: {...}) {
  const trimmed = (params.raw ?? "").trim();
  if (!trimmed) {
    return params.sessionScope === "global"
      ? "global"
      : buildAgentMainSessionKey({
          agentId: params.currentAgentId,
          mainKey: params.sessionMainKey,
        });
  }
  if (trimmed === "global" || trimmed === "unknown") return trimmed;
  if (trimmed.startsWith("agent:")) return trimmed.toLowerCase();
  return `agent:${params.currentAgentId}:${trimmed.toLowerCase()}`;
}
```

这个函数等于把用户输入补成规范形式。于是 TUI 内部就能同时支持“我只输一个裸 key”和“我明确指定 agent:key”两种用法。

实际操作时，Agent 选择、模型选择、会话选择都不是在一行里盲打完成，而是通过覆盖层（overlay）弹出 `SearchableSelectList` 。这是一种可搜索选择列表，交互体验很接近 VS Code 的命令面板：输入关键字、模糊匹配、回车确认。对 TUI 来说，这是很关键的一步，因为终端里可视空间小，靠 overlay 做“局部聚焦”比把所有选项铺满主界面要稳得多。

历史加载（history loading）也有一套固定策略。 `loadHistory()` 默认最多拉 200 条，随后按 transcript replay 的方式逐条回放：用户消息进用户气泡，助手消息进助手气泡，工具结果在 verbose

打开时重建成工具卡片。这样做的好处是，界面状态并不依赖“上一次渲染剩下了什么”，而是可以从持久化转录重新演算出来。这个性质很像事件溯源（event sourcing）的简化版。

41.2.5 本地 Shell 执行

`!` 前缀让 TUI 具备了一个很特别的能力：不离开聊天界面，直接在本机跑 Shell（Shell）命令。这个功能方便，但风险也高，所以 OpenClaw 给它上了三层保险。

第一层是前缀检查（prefix check）。只有输入第一字符真的是 `!`，而且不只是孤零零一个 `!`，才会走本地执行分支。第二层是一次性批准（one-time approval）。第一次执行时，TUI 会弹出确认覆盖层，明确提醒“命令跑在你的机器上，不是跑在 gateway 上”。这一关没过，本会话后续所有本地命令都会被拒绝。第三层是会话隔离（session isolation）：授权状态只在当前 TUI 会话里生效，不会自动扩散到别的终端实例。

底层执行用的是 Node.js 的 `spawn()`，并且显式设置 `shell: true`。工作目录来自 `getCwd()`，环境变量里还会附带 `OPENCLAW_SHELL="tui-local"`，方便后续做来源识别或审计。

```
const child = spawnCommand(cmd, {
  shell: true,
  cwd: getCwd(),
  env: { ...env, OPENCLAW_SHELL: "tui-local" },
});
```

输出处理也做了限制。实现里给标准输出和错误输出加了 40KB 上限，超过就从尾部截断保留最近内容，避免一个失控命令把 TUI 挤爆。进程结束后，界面会追加 `[local] exit <code>`，如果是被信号杀掉，还会带上 `signal`。这件事看着不起眼，但对排错非常关键：终端里能不能一眼看出“命令失败了还是只是没有输出”，差别很大。

41.2.6 键盘快捷键与输入优化

OpenClaw 的 TUI 交互并不只靠 Enter 提交。就当前这版源码来说，一组高频动作直接挂到了快捷键上，而且有些绑定和传统终端习惯不太一样：

快捷键	行为
`Ctrl+C`	有输入时清空；空输入时先警告；1000ms 内再按一次退出
`Ctrl+D`	直接退出 TUI
`Ctrl+L`	打开模型选择器，而不是清屏

快捷键	行为
`Ctrl+O`	展开或折叠工具输出
`Ctrl+P`	打开会话选择器
`Ctrl+G`	打开 Agent 选择器
`Ctrl+T`	切换是否显示 thinking，并重载历史
`Alt+Enter`	编辑器层预留的多行输入钩子
`Shift+Tab`	编辑器层预留的反向导航钩子

这里面最值得说的是 `Ctrl+C` 。很多 CLI 程序一按就退出，OpenClaw 没这么做。它先调用 `resolveCtrlCAction()` 判断当前输入框是不是有内容；有内容时，第一次按下是“清空输入”；没内容时，第一次是“警告”，第二次才真退出，而且窗口期只有 1000ms。这个设计很像图形界面里的“二次确认”，但比弹窗更轻，不打断思路。

输入优化里还有一个很聪明的细节：突发提交合并（burst coalescing）。有些终端在粘贴多行文本时，不会把整块内容一次性交给编辑器，而是拆成很多很快的单行提交。OpenClaw 用一个 50ms 的窗口把这些碎片重新并成一个多行消息。换句话说，它不是在“处理粘贴”，而是在识别一种粘贴的时间模式。

平台兼容性也考虑到了。Windows 下的 Git Bash（Git Bash），以及 macOS 下的 iTerm（iTerm）和 Terminal.app，都可能触发这种粘贴退化行为，所以代码里专门有 `shouldEnableWindowsGitBashPasteFallback()` 去打开回退逻辑。顺手补一句：设置面板（settings）在当前版本主要通过 `/settings` 打开，不走单独快捷键。TUI 做到这里，已经不只是“能跑”，而是开始认真处理终端生态里那些说不上高级、但很烦人的边角问题了。

本节小结

这一节可以浓缩成一句话：OpenClaw 的 TUI 之所以用起来顺，不是因为它会收事件，而是因为它把事件“整理”过了。聊天事件负责用户能看见的回答流，代理事件负责工具与生命周期细节； `sessionRuns` 、 `finalizedRuns` 和流组装器一起，撑起了去重、收口和刷新逻辑；斜杠命令、覆盖层选择器和会话解析函数，把复杂操作压缩成了几次击键；本地 Shell 执行则在易用性和安全性之间做了谨慎折中。到这里我们就能看清一件事：OpenClaw 的 TUI 不是给 Gateway 套了个终端皮肤，它本身就是一层认真设计过的人机交互适配层。